

MODUL PRAKTIKUM: OBJECT-ORIENTED PROGRAMMING

MINGGU 12: Exception Handling & Robustness

Waktu Pengerjaan: 100 menit (Lab Session)

Kata-kata pembuka:

"Hidup itu kayak program — error itu pasti ada, yang penting kamu tahu cara handle-nya. Jangan panik waktu ketemu bug, justru di situ karakter kamu dibentuk. Orang yang hebat bukan yang nggak pernah gagal, tapi yang tahu cara bangkit dan nulis catch block yang bener."

A. ATURAN PROJECT (REMINDER)

Ingat kebijakan Single Repository dari minggu lalu. Jangan buat project baru!

1. Buka project `IF433_Lab_Works` yang sudah dibuat minggu lalu.
2. Di dalam folder `src`, buat package baru `oop_<nim>_<nama>.week12`.
3. Pastikan semua kode hari ini masuk ke package `week12`.

B. LATIHAN TERBIMBING 1: Extension Functions

Tujuan: Memahami blok standar penanganan *error* dan memanfaatkan kemampuan Kotlin yang menjadikan `try` sebagai sebuah *expression* (mengembalikan nilai).

LANGKAH 1: try-catch-finally Dasar

1. Buat file baru bernama `BasicException.kt`.
2. Ketik kode berikut untuk mencoba pembagian dasar yang rawan terhadap `ArithmeticException`:

```
package oop_001_johnthor.week12

fun divide(a: Int, b: Int): Int {
    try {
        return a / b
    } catch (e: ArithmeticException) {
        println("Error: ${e.message}")
        return -1
    } finally {
        println("Division attempt finished")
    }
}
```

CHECKPOINT 1: Commit ke Git dengan pesan:

```
"week12: create basic try-catch-finally"
```

LANGKAH 2: **try sebagai Expression**

Di Kotlin, **try** bisa langsung mengembalikan nilai (mirip seperti **if-else**). Ini menghilangkan kebutuhan deklarasi variabel penampung bernilai **null** sebelum blok **try**.

1. Tambahkan fungsi **main** di file yang sama dan buat *expression*:

```
fun main() {  
    println("=== TEST TRY AS EXPRESSION ===")  
    val inputString = "123A"  
  
    // try langsung mengembalikan hasil parsing, atau -1 jika gagal  
    val result: Int = try {  
        Integer.parseInt(inputString)  
    } catch (e: NumberFormatException) {  
        -1  
    }  
  
    println("Hasil parsing: $result")  
}
```

CHECKPOINT 2: Commit dengan pesan:

```
"week12: implement try as expression"
```

C. LATIHAN TERBIMBING 2: **throw & Custom Exceptions**

Tujuan: Membuat *exception* kustom untuk merepresentasikan kendala yang spesifik terhadap bisnis logika kita (domain-specific errors) dan membangkitkannya menggunakan keyword **throw**.

LANGKAH 1: Membuat Custom Exception

1. Buat file baru bernama **CustomExceptions.kt**.
2. Kita akan membuat pengecualian untuk saldo yang tidak cukup. Ingat, *best practice* di Kotlin adalah menurunkan kelas dari **Exception** untuk *error* yang bisa di-*recover*.

```
package oop_001_johnthor.week12

// Custom exception dengan membawa data tambahan
class InsufficientFundsException(
    val amount: Double,
    val balance: Double
) : Exception("Attempted $amount, balance: $balance")
```

CHECKPOINT 3: *Commit* dengan pesan:

"week12: create InsufficientFunds exception"

LANGKAH 2: Menggunakan throw untuk Validasi

1. Buat kelas *Bank Account* sederhana di file yang sama dan lemparkan *exception* jika logika dilanggar:

```
class BankAccount(var balance: Double) {
    fun withdraw(amount: Double) {
        if (amount < 0) {
            throw IllegalArgumentException("Amount must be positive")
        }
        if (amount > balance) {
            throw InsufficientFundsException(amount, balance)
        }
        balance -= amount
        println("Withdrawal successful. Remaining balance: $balance")
    }
}
```

CHECKPOINT 4: *Commit* dengan pesan:

"week12: implement throw keyword for validation"

LANGKAH 3: Multiple Catch Blocks

1. Mari buat blok `main()` untuk menangkap lebih dari satu jenis kesalahan secara spesifik. Ingat, tangkap *exception* yang paling spesifik terlebih dahulu!

```

fun main() {
    println("=== TEST MULTIPLE CATCH ===")
    val account = BankAccount(100.0)

    try {
        account.withdraw(150.0) // Ini akan memicu InsufficientFundsException
    } catch (e: InsufficientFundsException) {
        println("Caught Domain Error: Uang tidak cukup. ${e.message}")
    } catch (e: IllegalArgumentException) {
        println("Caught Argument Error: Input tidak valid. ${e.message}")
    } catch (e: Exception) {
        println("Caught General Error: Terjadi kesalahan tidak terduga.")
    }
}

```

CHECKPOINT 5: *Commit* dengan pesan:

"week12: implement multiple catch blocks"

D. LATIHAN TERBIMBING 3: The runCatching Idiom

Tujuan: Mengubah gaya penanganan *error* konvensional menjadi fungsional dan terkomposisi menggunakan `runCatching` dan API `Result<T>`.

LANGKAH 1: Penggunaan runCatching Dasar

1. Buat file `RunCatchingExample.kt`.
2. Alih-alih membungkus dengan `try-catch` eksplisit, kita bungkus dalam `runCatching`:

```

package oop_001_johnthor.week12

fun main() {
    println("=== TEST RUNCATCHING ===")
    val result: Result<Int> = runCatching {
        "42X".toInt()
    }
}

```

CHECKPOINT 6: *Commit* dengan pesan:

"week12: implement basic runCatching"

LANGKAH 2: Menggunakan `getOrElse` dan `recover`

1. Tambahkan kode berikut untuk memberikan nilai *default* apabila operasi di dalam `runCatching` mengalami kegagalan:

```
// Pattern: getOrElse
val safeValue = result.getOrElse { -1 }
println("Safe Value (getOrElse): $safeValue")

// Pattern: recover (bisa mengubah tipe kegagalan jadi sukses)
val recovered = result.recover { 0 }.getOrNull()
println("Recovered Value: $recovered")
```

CHECKPOINT 7: *Commit* dengan pesan:

"week12: handle Result using getOrElse and recover"

LANGKAH 3: Chaining `onSuccess` & `onFailure`

1. Tambahkan *chaining* untuk menjalankan aksi spesifik berdasarkan status keberhasilan `Result`:

```
runCatching {
    "100".toInt()
}.onSuccess { v ->
    println("Berhasil dikonversi: $v")
}.onFailure { e ->
    println("Gagal konversi: ${e.message}")
}
```

CHECKPOINT 8: *Commit* dengan pesan:

"week12: implement onSuccess and onFailure chaining"

E. TUGAS MANDIRI: Smart Pet Feeder Robustness

Skenario: Anda sedang mengembangkan sistem perangkat lunak untuk "Smart Auto-Feeder" peliharaan. Peliharaan kesayangan Anda sangat bergantung pada alat ini untuk jadwal makan *kibble* hariannya. Sistem ini harus kebal terhadap masalah mekanik, seperti stok makanan yang

habis atau motor penampung/dispenser yang tiba-tiba tersangkut (*jammed*), agar aplikasi tidak *crash* dan tetap bisa memberikan notifikasi ke pemiliknya.

Gunakan *Custom Exceptions* dan fungsionalitas `runCatching` untuk membangun *pipeline* eksekusi yang aman.

Langkah-langkah Tugas:

1. Hierarchy Exception (Sealed Class):

Buat file `FeederExceptions.kt`. Buat sebuah `sealed class`
`FeederException(msg: String) : Exception(msg).`

CHECKPOINT 9: Commit dengan pesan:

```
"week12: (task) create FeederException sealed class"
```

2. Custom Exceptions Subclasses:

Di file yang sama, buat dua *exception* yang mewarisi `FeederException`:

- `class FoodEmptyException(requested: Int, available: Int) :
FeederException("Kibble tidak cukup! Diminta $requested gr, sisa
$available gr")`
- `class DispenserJamException : FeederException("Wadah dispenser
tersangkut/macet!")`

CHECKPOINT 10: Commit dengan pesan:

```
"week12: (task) create custom feeder exception subclasses"
```

3. Feeder Engine Setup:

Buat file `SmartFeeder.kt`. Buat fungsi `fun dispenseKibble(requestedGram: Int, availableGram: Int, isJammed: Boolean): Int`. Fungsi ini bertugas mengembalikan sisa stok kibble setelah dikurangi porsi yang diminta.

- Gunakan fungsi bawaan Kotlin `require(requestedGram > 0) { "Porsi kibble harus lebih dari 0 gr" }` di baris pertama fungsi.

CHECKPOINT 11: Commit dengan pesan:

```
"week12: (task) implement dispenseKibble function"
```

4. Validasi Dispenser (Hardware Check):

Di dalam `dispenseKibble`, periksa jika `isJammed` bernilai `true`. Jika ya, gunakan keyword `throw` untuk melempar `DispenserJamException`.

CHECKPOINT 12: Commit dengan pesan:

```
"week12: (task) implement hardware validation with throw"
```

5. Validasi Stok Makanan:

Masih di dalam `dispenseKibble`, periksa jika `requestedGram > availableGram`. Jika ya, lempar `FoodEmptyException`. Jika semua aman, cetak "Kibble berhasil dikeluarkan!" dan `return availableGram - requestedGram`.

CHECKPOINT 13: Commit dengan pesan:

```
"week12: (task) implement food stock validation with throw"
```

6. Main System Setup:

Di dalam file `SmartFeeder.kt`, buat fungsi `main()`. Buat variabel awal `var currentKibbleStock = 50`.

CHECKPOINT 14: Commit dengan pesan:

```
"week12: (task) setup SmartFeeder main function"
```

7. Jadwal Makan 1: Eksekusi dengan Multiple Catch:

Lakukan simulasi jadwal makan pagi. Masukkan ke dalam blok `try-catch`. Panggil `dispenseKibble(requestedGram = 80, availableGram = currentKibbleStock, isJammed = false)`. Sediakan blok `catch` secara berurutan untuk: `DispenserJamException`, `FoodEmptyException`, dan `Exception` umum.

CHECKPOINT 15: Commit dengan pesan:

```
"week12: (task) handle feeding execution with multiple catch"
```

8. Jadwal Makan 1: Cleanup (Finally):

Tambahkan blok `finally` di eksekusi Jadwal 1 yang mencetak "Siklus pengecekan dispenser pagi selesai."

CHECKPOINT 16: Commit dengan pesan:

```
"week12: (task) implement finally block for feeder logging"
```

9. Jadwal Makan 2: Eksekusi dengan runCatching:

Pemilik baru saja mengisi ulang alat sehingga alat normal. Lakukan simulasi jadwal makan sore menggunakan `runCatching` secara fungsional. Panggil eksekusi yang valid: `runCatching { dispenseKibble(requestedGram = 30, availableGram = 1000, isJammed = false) }`.

CHECKPOINT 17: Commit dengan pesan:

```
"week12: (task) execute feeding using runCatching"
```

10. Jadwal Makan 2: Handling Success:

Lakukan *chaining* hasil operasi di atas dengan `.onSuccess { newStock -> ... }`. Di dalamnya, perbarui nilai `currentKibbleStock = newStock` dan cetak "Makan sore sukses! Sisa stok kibble: \$currentKibbleStock gr".

CHECKPOINT 18: Commit git dengan pesan:

```
"week12: (task) handle feeding success using onSuccess"
```

11. Jadwal Makan 2: Handling Failure:

Lanjutkan *chaining* dengan `.onFailure { error -> ... }`. Cetak pesan kegagalan secara terformat: `"Peringatan ke Pemilik: ${error.message}"`. Sebagai bonus camilan karena alat rusak/stok habis, Anda bisa mencetak log `"(Opsional: Berikan chicken jerky secara manual)"`.

CHECKPOINT 19: Commit git dengan pesan:

```
"week12: (task) handle feeding failure using onFailure"
```

12. Final Run:

Jalankan program bot Anda. Pastikan tangkapan error pada Jadwal 1 (karena porsi diminta 80 tapi stok hanya 50) bekerja masuk ke `FoodEmptyException`, blok *finally* tereksekusi, dan Jadwal 2 berhasil dijalankan dengan mulus lewat pipeline Result API.

CHECKPOINT 20: Commit final dengan pesan:

```
"week12: (task) test full robust smart feeder pipeline"
```


G. PENGUMPULAN

1. **Push ke GitHub:** Pastikan 20 Checkpoint (Commit) di atas sudah terkirim. `git push`
2. **Validasi:** Cek tab "Commits" di *repository* GitHub Anda. Riwayat pembuatan `try-catch` konvensional hingga *pipeline* `runCatching` untuk Smart Feeder harus terlihat berurutan sesuai checkpoint.
3. **Submit:** Masukkan tautan repositori GitHub ke e-learning UMN sesuai instruksi sebelumnya.

Kata-kata penutup:

"Exception itu kayak kegagalan dalam hidup — nggak bisa kamu hindari sepenuhnya, tapi bisa kamu handle dengan elegan. Yang penting bukan seberapa jarang kamu nulis `catch`, tapi seberapa cerdas kamu handle setiap error yang muncul. Keep debugging, keep growing!"